Fall Semester 2026
Foundations of Distributed Systems
Part I: Communication
Lecturer: Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch, Office 06.002)
Assistant: Ali Ajorian (ali.ajorian@unibas.ch)
Topics: Vector Clocks, gRPC, Consensus and CRDT

Guidelines

This exercise consists of four tasks. The first task involves an investigation of vector clocks. The second focuses on working with gRPC, a remote procedure call framework. The third requires an investigation and implementation related to the consensus problem in distributed systems. The fourth focuses on applying Conflict-Free Replicated Data Types (CRDTs) to achieve eventual consistency.

Regarding organizational details, please note the following:

- None of the tasks are mandatory, and therefore no submission is required. You are free to form groups and work on all four tasks jointly.
- Two exercise sessions are reserved for this exercise on Tuesday, 29.09.2026, and Tuesday, 06.10.2026, from 10:15 to 12:00 in Pharmazentrum, Seminarraum U1075.
- An EvaExam will be held on Tuesday, 20.10.2026, from 10:15 to 12:00 in Pharmazentrum, Seminarraum U1075. The exam will cover all theoretical concepts and implementation aspects of this exercise.
- During the first exercise session on 29.09.2026, further details about the EvaExam will be discussed in the first part of the session with all four teaching assistants.

Important: Although completing the exercises is not mandatory, **participation in the EvaExam is mandatory**. To be eligible to participate in the final course exam, you must earn at least 50 percent of the points in each EvaExam and at least 75 percent of the total points across all four EvaExams.

Fall Semester 2026
Foundations of Distributed Systems
Part I: Communication
Lecturer: Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch, Office 06.002)
Assistant: Ali Ajorian (ali.ajorian@unibas.ch)
Topics: Vector Clocks, gRPC, Consensus and CRDT

Task 1: Vector Clocks

In this task, you will explore various methods of representing the causality of events within a distributed system. In Git context, each event (or commit) has a unique hash value that serves as an *event ID*. These event IDs are used as pointers within each commit object to reference other commit objects. This creates a Directed Acyclic Graph (DAG) data structure that illustrates the causal relationships between commits. On the other hand, vector clocks are vectors of N logical clocks that can determine the partial ordering of events in a distributed system with N processes. This means that these mathematical objects convey the causality between events without relying on edges as a DAG does. By doing this task, you will see how both DAGs and vector clocks convey and represent causality between events using different mechanisms.

Data Representation

For this task, we will represent Git DAGs using a JSON file formatted as follows:

```
{ "Branch_Name" : {"Commit_Name": list of parent commits}}
```

For example, the following JSON data represents a Git repository with three branches: B1, B2, and B3. In this structure, the commit [1] object 1111 in branch B1 has no parent, indicating it is the initial commit. Similarly, the commit object e13b in branch B3 has two parents, 9634 and f432. The accompanying DAG illustrates the complete causal relationships derived from this JSON data.

JSON Representation

```
{ "B1":
    {
        "1111":[],
        "12f3":["1111"],
        "f432": ["12f3","2101"]
    },
    "B2":
    {
        "2101":["1111"]
    },
    "B3":
    {
        "9634":["2101"],
        "e13b":["f432","9634"]
    }
}
```

DAG Representation

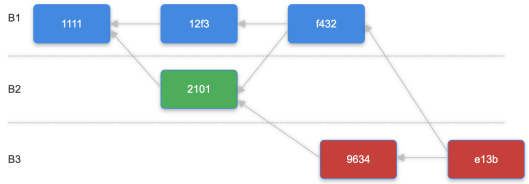


Figure 1: A Git repository represented by a DAG and a json file

[1] For simplicity, we have only written the last four digits of all hash values

Having the JSON file above, we can consider each branch as a process and each commit as an event. This allows us to calculate vector clocks for the commits, as illustrated in the following image.

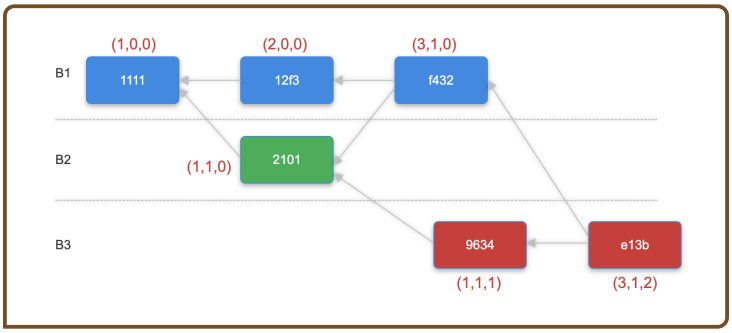


Figure 2: Vector clocks for the DAG represented above

Your Task

1. Given a JSON file representing a Git DAG as outlined above, write a Python script to calculate the vector clocks for each event, treating branches as processes and commits as events. You can store these values internally in a Python dictionary, as this will be helpful for subsequent tasks. However, the output of your function must be a JSON file formatted as follows:

   ```
   { "Commit_Name": list of clock values}
   ```

   The output of your function for the previous DAG should be formatted as follows:

   {"1111":[1,0,0], "12f3":[2,0,0], "f432": [3,1,0], "2101":[1,1,0], "9634":[1,1,1], "e13b":[3,1,2] }

2. Vector clocks establish a partial order relation among events. Implement a function named *causally_precedes(a,b)* that takes two vector clocks values, a and b, as input and determines whether the first vector clock precedes the second. Utilize this function and the output of section 1 to generate a graph where all nodes with causal relationships are interconnected.

3. Some edges (causal relations) in the graph plotted in section 2 are redundant due to the transitive nature of causality. Develop a function that visualizes a graph with the minimum number of edges necessary to represent the partial order relation between nodes. To plot graphs you can use either Python libraries or graphviz utility. In the second case you should write a python script which generates a .dot file for each target graph.

The JSON file in Figure 1 serves as a concrete example. Ensure that your implementation correctly handles different JSON files with similar structure.

Lecturer: Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch, Office 06.002)
Assistant: Ali Ajorian (ali.ajorian@unibas.ch)

Task2: Remote Procedure Call

In this task, you will explore gRPC, an open-source Remote Procedure Call (RPC) framework created by Google. It provides support for multiple programming languages, including C++, Java, Python, Go, Ruby, and Node.js, ensuring high performance and efficiency. To maintain language independence, gRPC utilizes Protocol Buffers (protobuf) as its interface definition language (IDL). This enables you to define the data structures and available methods for your service in a .proto file.

Theory of the Task

In today's distributed systems, it is common to delegate limited access to user data without exposing passwords to third-party services or applications. For instance, a user can register on a server, store their data in its database, and grant a third-party application access to certain parts of that data. To achieve this, various standards such as OAuth exist. However, for this task, we will create a simple protocol and implement it using gRPC as shown in Figure 3.

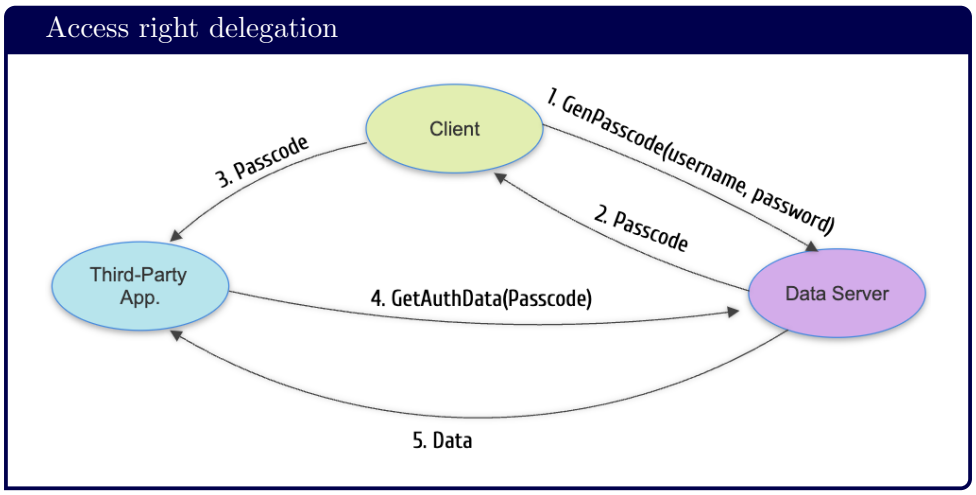


Figure 3: A user can grant access to their data using a passcode.

In our design, different users can register and store their text data in a *Data Server*. This server provides the following services:

1. A user can register on the server by providing a username and a password.
2. A user can store their data on the server by providing their username, password, and text data.
3. A user can retrieve their data from the server by providing their username and password.
4. A user can request the server to generate a one-time use passcode by providing their username and password.

5. A user or a third-party application can retrieve the user data by providing the one-time use passcode.

Having the features 4 and 5, a user can delegate their access to a third-party application. They must request the server to generate a passcode, which serves as a delegation tool, and then send this generated passcode to the third-party application. With the passcode in hand, the third-party application can retrieve the user data from the Data Server, effectively acting as the user's agent.

Target Scenario

As shown in Figure 4, our target scenario involves a third-party application that computes hash values for client data stored on a data server. The numbered steps in the figure indicate the sequence of operations. All components run locally: the data server on port 50051, the hash server on port 50052, and the client orchestrating the scenario. The data server services are already implemented in Node.js, while you must implement both the hash server services and the client using Python 3.

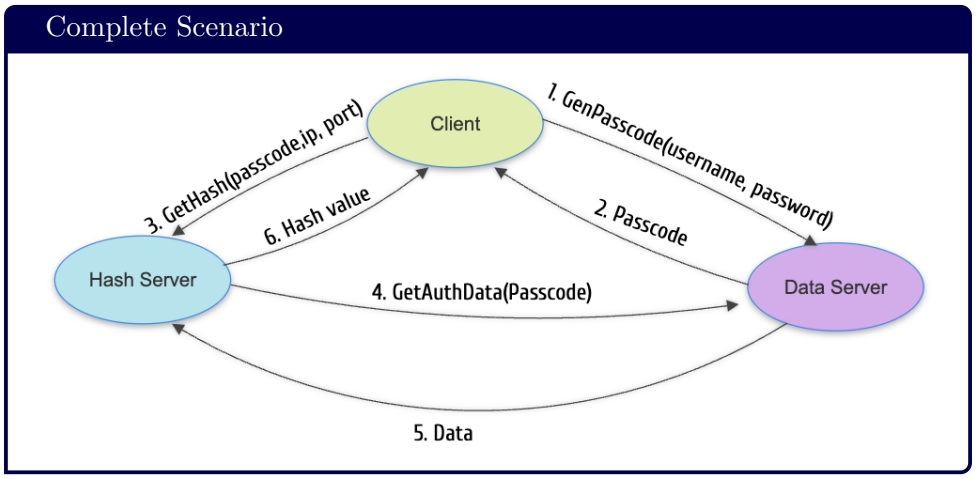


Figure 4: A third-party server can calculate Hash value of user's data

1. Data Server

The data server is pre-implemented in Node.js within the file *dataServer.js*, providing the gRPC data services described in the "Theory" section. The source code is located in the *dataServer* directory of the attached template.
To run the server:

1. Ensure Node.js is installed on your machine.
2. Install the required gRPC dependencies by running the following command in the *dataserver* directory:

```
npm install @grpc/grpc-js @grpc/proto-loader
```

3. Execute the following command to start the data server:

```
node dataServer.js
```

Upon successful startup, you should see a confirmation message:

`"Data server listening on port 50051".`

The data server implements the gRPC services defined in *dservice.proto* file:

```
syntax = "proto3";

package DATA;

message Result {bool success = 1;}
message Data {string msg = 1;}
message StoreReq{string username =1; string password = 2; string msg = 3;}
message Passcode {string code = 1;}
message UserPass{ string username = 1; string password = 2;}

service DB
{
    rpc RegisterUser(UserPass) returns (Result);
    rpc StoreData(StoreReq) returns (Result);
    rpc GenPasscode(UserPass) returns (Passcode);
    rpc GetData(UserPass) returns (Data);
    rpc GetAuthData(Passcode) returns (Data);
}
```

The services are as follows:

- **RegisterUser**: Takes a `UserPass` message (username/password pair) to register a new client on the data server.
- **StoreData**: Takes a `StoreReq` message (username, password, and data) allowing registered clients to store their data on the server.
- **GenPasscode**: Enables registered clients to generate a one-time passcode by providing their username and password. This passcode can be delegated to third-party applications (like the hash server) for temporary data access via `GetAuthData`. The passcode expires automatically after use.
- **GetData**: Allows clients to retrieve their stored data using their username and password.
- **GetAuthData**: Enables third-party applications to retrieve user data using a valid passcode obtained from the client.

2. Hash Server

The hash server provides a gRPC service for calculating hash values of text data. This service is used by clients who have previously registered with the data server, stored their data, and generated a passcode to delegate read access to the hash server. The service interface is defined

as follows:

```
syntax = "proto3";
package Hash;

message Response {string hash = 1;}
message Request {string passcode =1;string ip=2; uint32 port = 3;}

service HS
{
    rpc GetHash(Request) returns (Response);
}
```

The **GetHash** procedure takes three parameters:

- A **passcode** generated by the data server
- The **IP address** of the data server (in our local scenario: `127.0.0.1`)
- The **port number** on which the data server is listening (port `50051`)

Using the provided passcode, the hash server retrieves the user's data from the data server, calculates its hash value, and returns the result to the user.

3. Client

The client performs the following sequence of operations:

1. Registers with the data server by remotely calling the `RegisterUser` procedure.
2. Stores data on the data server by remotely calling the `StoreData` procedure.
3. Generates a passcode by remotely calling the `GenPasscode` procedure on the data server.
4. Requests hash computation by remotely calling the `GetHash` procedure on the hash server, providing the obtained passcode.

Your Task

1. Run the data server on your machine.
2. Implement a Python server that provides the hash service defined in the specified gRPC prototype and run it.
3. Create a Python client that registers you on the *Data Server* using your chosen username and password. Store your text data on the *Data Server*.
4. Request the *Data Server* to generate a passcode. After obtaining this passcode, connect to the *Hash Server* and retrieve the hash value of your data and output it.

gRPC on Python

Installation

To install gRPC in your Python environment, use the following commands, or appropriate alternatives compatible with the version of Python installed on your machine:

```
pip3 install grpcio grpcio-tools
pip3 install protobuf==5.27.2
```

To implement the client code and the hash server in Python, you need to compile the data server and hash server prototype files (*dservice.proto* and *hservice.proto*) into Python files. The following commands handle this compilation:

```
python -m grpc_tools.protoc -I. --python_out=.  --grpc_python_out=.  hservice.proto
python -m grpc_tools.protoc -I. --python_out=.  --grpc_python_out=.  dservice.proto
```

These commands generate the following Python files:

- *dservice_pb2py* and *hservice_pb2.py*: These contain the Python class definitions for our protocol buffer *messages* (like Request, Response, UserPass). They are used to create, serialize, and deserialize the data structures you send and receive.
- *dservice_pb2_grpc.py* and *hservice_pb2_grpc.py*: These contain the definitions for the gRPC clients (stubs) and servers (services). The **_grpc.py‘* files provide the necessary classes to call remote procedures on the server and to implement the server-side logic that handles those calls.

In short, the *_pb2.py* files handle your data, while the *_pb2_grpc.py* files handle the communication for their respective services.

Communication Basics

To interact with a gRPC server (such as the data server or hash server), first establish a connection by creating an insecure channel to the server's address and port, as shown in the following code snippet. This channel serves as the communication pipeline. Next, instantiate a stub (client) for the specific service you want to use; this stub provides local methods that correspond to the remote procedures defined in your *.proto* file.

```python
import grpc
import hservice_pb2
import hservice_pb2_grpc
import dservice_pb2
import dservice_pb2_grpc

ip = "localhost"
port = 50051
channel = grpc.insecure_channel(f'{ip}:{port}')
d_stub = dservice_pb2_grpc.DBStub(channel)
```

To call a remote procedure, invoke the corresponding method on the stub, passing a properly constructed request message. For example, to call the *RegisterUser* procedure on the data server from the client, use the *d_stub* object as shown below:

```
usrpss = dservice_pb2.UserPass(username='John', password='Hello')
res = d_stub.RegisterUser(usrpss)
if not res.success: print('failure')
```

When a call is received by a server, the corresponding method is automatically executed, and whatever value it returns is sent back as the response to the client.

To start up a gRPC server so that clients can actually use its service, *grpc* module has a class *server* that can be used as follows:

```
    srv = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    route_guide_pb2_grpc.add_RouteGuideServicer_to_server(RouteGuideServicer(), srv)
    srv.add_insecure_port(f"[::]:{port}")
    srv.start()
    srv.wait_for_termination()
```

Lecturer: Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch, Office 06.002)
Assistant: Ali Ajorian (ali.ajorian@unibas.ch)

Task 3: Consensus Algorithm

In this task, you will practice implementing a simple consensus algorithm using a simulated distributed system. Consensus algorithms are crucial in distributed systems to ensure that multiple nodes agree on a single data value, even in the presence of failures. Understanding these algorithms is fundamental for building resilient and reliable systems.

We have provided a framework that simulates a distributed system of N nodes, where each node is simulated by a separate thread. As these threads execute independently, they mimic the behavior of distinct computers in a distributed system. Communication between nodes is simulated using a globally shared buffer, which acts as the network channel. Nodes send messages by writing to this buffer and receive messages by reading from their designated section within it. Although it is necessary to use synchronization mechanisms to prevent race conditions, we have considered small values of N for simplicity, where the likelihood of such issues is low enough for our pedagogical goals. Therefore, we have not incorporated synchronization primitives like locks or semaphores in our template code.

Template Code

The template code provided with this exercise contains our simulator implementation. As shown in Listing 1, the *initialize* function takes the number of nodes N as input, initializes the global list *nodes* with an instance of the *Node* class for each one, and then calls their *start* method to begin their individual threads. The global *buffer* object is also declared to store the simulated messages exchanged between nodes. It is implemented as a dictionary that maintains a list of **received** messages for each node. The dictionary's keys are the node IDs (integers from 0 to $N-1$). Each message is a tuple containing the data and a message type, allowing you to distinguish between different kinds of received messages.

Listing 1: Template Code: Part I

```
import threading
import time
nodes = []
buffer = {}

def initialize(N):
    global nodes
    nodes = [Node(i) for i in range(N)]
    for node in nodes:
        node.start()
```

As shown in Listing 2, the class *Node* implements the behavior of each node in the simulator. Its constructor *__init__* takes the node's ID as input and registers its dedicated message queue in the global buffer. The *start* function, as mentioned previously, launches a new thread for the node. This thread executes the *run* function. The *run* function consists of a busy-waiting loop that periodically checks the node's queue in the global buffer. When a message is received, it is popped from the buffer and delivered to the node by calling the *deliver* method. A part of your task is to implement the logic inside the *deliver* function to handle different types of received messages. Regarding the *run* function, the following point is critical:

All functions called from within run are executed in the node's own thread. Therefore, every

behavior of a node must be triggered from its thread. This means you can either write the logic directly inside the run function or implement it in a separate function that is called by run.

Additionally, a helper function named *broadcast* is provided. It takes a message type and its corresponding data, then broadcasts a tuple $(type, data)$ to all nodes by placing it into each node's respective queue in the buffer object.
The framework also supports node crashes, which disconnects a node from the network. The underlying cause, hardware or network failure, is treated as an implementation detail outside the scope of this model. Two functions, *crash* and *recover*, are provided to trigger the failure and subsequent recovery of a node, respectively.

Listing 2: Template Code: Part II

```
class Node:
    def __init__(self,id):
        buffer[id] = []
        self.id = id
        self.working = True
        self.state = 'unknown'

    def start(self):
        print(f'node {self.id} started')
        threading.Thread(target=self.run).start()

    def run(self):
        while True:
            while buffer[self.id]:
                msg_type, value = buffer[self.id].pop(0)
                if self.working: self.deliver(msg_type,value)
            time.sleep(0.1)

    def broadcast(self, msg_type, value):
        if self.working:
            for node in nodes:
                buffer[node.id].append((msg_type,value))

    def crash(self):
        if self.working:
            self.working = False
            buffer[self.id] = []

    def recover(self):
        if self.working:
            buffer[self.id] = []
            self.working = True

    def deliver(self, msg_type, value):
        pass
```

Consensus Algorithm Simulation

We implement a simple leader election algorithm as an instance of a consensus problem within our simulator, assuming that all the nodes are honest and do not behave byzentine. In leader election, each node can be in one of three states: follower, candidate, or leader.

- **Leader:** A node that has been elected by a majority during an election process.

- **Follower:** A node that follows the leader's decisions.
- **Candidate:** A temporary role a node assumes when there is no elected leader or when the current leader is presumed to have crashed.

To detect the crash of an elected leader, a *heartbeat* mechanism is used. The leader periodically sends 'heartbeat' messages. By receiving these heartbeats, followers confirm the leader is still active. If a follower does not receive a heartbeat for a specified timeout period, it infers that the leader has crashed.

Your task

Your task is to implement the leader election process with the following assumptions.

- Each node starts in the 'follower' state by default. When a node detects there is no leader or that the current leader has crashed, it decides to become a candidate and start a new election process.
- The candidate starts the election process by broadcasting its ID to the network as a candidacy message. Since multiple followers may detect the need for a new election simultaneously, a potential candidate waits randomly for 1 to 3 seconds before broadcasting its candidacy to avoid simultaneous elections. During this waiting period, if the node receives a candidacy message from another node, it resigns its candidacy by not sending its own candidacy message.
- If no other candidacy is received during the waiting period, the candidate broadcasts its candidacy message and enters a vote collection period of 2 seconds. After this period, it counts the total received votes. If it has obtained more than half of the votes, it becomes the leader, changes its state to 'leader', and starts broadcasting heartbeats.
- When any node receives a candidacy message, regardless of whether it had detected the need for an election, if it has not yet voted, it will vote for the candidate by broadcasting a vote message containing both its voter ID and the candidate's ID. Note that each candidate votes for itself when announcing its candidacy.
- During the entire leader election process, if any node receives a heartbeat, it stops the election process and accepts the heartbeat sender as the leader. The leader sends heartbeat messages every 0.5 seconds. If a node does not receive a heartbeat for one second, it infers that the leader has crashed.

In the following box, you will see a scenario involving 3 nodes executing the election algorithm, with node '0' being elected as the leader:

```
node 0 started
node 1 started
node 2 started
node 0 is starting an election.
node 0 voted to node 0
node 1 voted to node 0
node 1 detected node 0 as leader
node 0 detected node 0 as leader
node 2 voted to node 0
node 2 got a heartbeat and followed node 0 as leader
$ state
       node 0: leader
       node 1: follower
       node 2: follower
```

Lecturer: Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch, Office 06.002)
Assistant: Ali Ajorian (ali.ajorian@unibas.ch)

Task4: CRDT

As you have seen in the lecture, Causal Length Set, CLS, allows adding and removing set elements without any coordination, yet the final state of the set in a decentralized context. Such a CRDT allows us to implement a shared shopping cart **without** servers or consensus algorithm! Image that Alice and Bob have a shared shopping list for the next grocery purchases. While on the road they update their shopping list and intermittently their devices synchronize with each other. The nice property of CLS is that items can be added and be removed several times by any party without coordination, yet all parties eventually agree on the final state.
Your task is to implement a CLS class in Python such that the following system trace can be executed.

Listing 3: A simulated shared shopping cart

```
alice_list = CLS()
bob_list = CLS()

alice_list.add('Milk')
alice_list.add('Potato')
alice_list.add('Eggs')

bob_list.add('Sausage')
bob_list.add('Mustard')
bob_list.add('Coke')
bob_list.add('Potato')
bob_list.mutual_sync([alice_list])

alice_list.remove('Sausage')
alice_list.add('Tofu')
alice_list.remove('Potato')
alice_list.mutual_sync([bob_list])

print("Bob's list contains 'Potato' ?", bob_list.contains('Potato'))
```

Lecturer: Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch, Office 06.002)
Assistant: Ali Ajorian (ali.ajorian@unibas.ch)
Topics: Vector Clocks, gRPC, Consensus and CRDT

References and Useful Links

- gRPG: A high performance, open source universal RPC framework.
- matplotlib: a very powerful library for visualization with Python.
- graphviz: a open source visualization software supporting visualization *language dot*.
- nodejs: an open-source, cross-platform JavaScript runtime environment.
- OAuth: an industry-standard protocol for authorization.
- Consensus: agree on some data value that is needed during computation in a distributed system.
- CRDT: conflict-free replicated data type.